

AtlasLegalAI: A Legal AI Chatbot for Moroccan Law

Using RAG and Fine-Tuned Large Language Models

Boukaidi Tarik¹, BEL EL YAZID Ilyas², Daoudi Moaad³, Chahba Brahim⁴,
Cherradi Mohammed⁵

Digital Transformation and Artificial Intelligence
National School of Applied Sciences of Al Hoceima
May 2026

Abstract

AtlasLegalAI is a specialized legal chatbot focused on Moroccan law, designed to improve access to legal information using large language models (LLMs) and parameter-efficient fine-tuning (PEFT). The system is trained on a curated dataset of digitized Moroccan legal codes sourced from official government platforms. It fine-tunes the Qwen2.5-3B model using Low-Rank Adaptation (LoRA) with the Unsloth framework on CUDA-enabled GPU infrastructure, enabling efficient domain adaptation under limited computational resources.

To improve factual accuracy and reduce hallucinations, AtlasLegalAI integrates a Retrieval-Augmented Generation (RAG) pipeline based on FAISS, combined with the *intfloat/multilingual-e5-large* embedding model for semantic retrieval of relevant legal passages. The system supports bilingual interaction in French and Arabic, reflecting the linguistic structure of Moroccan legal documentation.

AtlasLegalAI is deployed as a full-stack web application using a React frontend and FastAPI backend, providing an accessible interface for end users. The proposed system demonstrates the effectiveness of combining PEFT-based fine-tuning with retrieval-augmented architectures for domain-specific legal applications, and offers a reproducible and efficient framework for building multilingual legal AI systems.

Index Terms — Legal Chatbot, Moroccan Law, Deep Learning, Large Language Models, LLM Fine-Tuning, Qwen2.5, LoRA, PEFT, RAG, FAISS, NLP, Bilingual AI, GPU Inference, Unsloth.

Table of Contents

I. Introduction	3
II. Related Work	3
III. Methodology.....	4
A. Data Collection and Processing	4
III. Training	7
1. Objective and Role in the AtlasLegalAI Pipeline	7
2. Dataset Evolution and Motivation for a Second Training Run.....	7
3. Model Selection and Fine-Tuning Configuration	8
4. Standalone Evaluation Protocol	9
5. Results.....	9
6. Discussion and Connection to the RAG Component.....	11
IV. Retrieval-Augmented Generation (RAG)	11
1. Retrieval-Augmented Generation	11
2. Methodology	12
2.1 System Architecture Overview.....	12
2.2 Base Model and Fine-Tuning	12
2.3 Legal Knowledge Base Construction	13
2.4 Embedding and Vector Store.....	14
2.5 RAG Inference Pipeline	14
2.6 Experimental Protocol.....	15
3. Results.....	15
3.1 Evaluation Summary Table	15
3.2 Limitations.....	16
V. Web Application Development	16
1. Architecture.....	16
2. Frontend Implementation.....	16
3. Backend Infrastructure and RAG Pipeline.....	17
4. Data Persistence and Privacy	17
VI. Experimental Results.....	19
VII. Discussion	19
VIII. Conclusion	19
Acknowledgements	19
References	20
Authors	20

I. Introduction

Access to legal information remains a profound challenge across Morocco and the broader Arab world. Despite significant advances in digital governance, the vast majority of citizens lack the practical means to navigate their own legal systems. Legal texts in Morocco are predominantly published in French — a language not fully accessible to all segments of the population — while Arabic-language resources remain fragmented and seldom digitized in machine-readable form. The consequence is a deep asymmetry: those who can afford professional legal counsel are protected by the law, while those who cannot are, in effect, excluded from it. Rural communities and low-income households are disproportionately affected, facing opaque administrative procedures and a body of legislation they have little practical ability to consult or understand.

The emergence of large language models (LLMs) offers a historically unprecedented opportunity to narrow this gap. Modern LLMs can read, reason over, and generate fluent explanations of complex normative texts in multiple languages, potentially serving as round-the-clock legal information assistants accessible from a smartphone. However, deploying such systems reliably in a high-stakes legal domain requires both domain adaptation and mechanisms to prevent factual hallucination — challenges that recent advances in parameter-efficient fine-tuning and retrieval-augmented architectures have begun to address, as surveyed in Section II.

Despite progress in Arabic NLP and legal AI, existing approaches are not tailored to the Moroccan legal context, which combines multilingual sources (French and Arabic), a civil-law tradition, and severe infrastructure constraints. These limitations motivate AtlasLegalAI.

We present AtlasLegalAI, a specialized legal chatbot built on the Qwen2.5-3B language model fine-tuned via LoRA on a curated bilingual corpus of Moroccan legal codes, combined with a FAISS-indexed RAG pipeline for authoritative retrieval at inference time. The remainder of this paper is organized as follows: Section II surveys related work in legal AI, parameter-efficient fine-tuning, and Arabic NLP; Section III details the full methodology; Section IV describes the web application architecture; Section V presents experimental results; Section VI discusses findings and limitations; and Section VII concludes with directions for future work.

II. Related Work

The application of artificial intelligence to legal information retrieval and question answering has attracted growing interest over the past decade. Early work such as DoNotPay demonstrated the feasibility of rule-based legal chatbots for simple consumer disputes, while ROSS Intelligence introduced the first commercial AI-powered legal research platform based on IBM Watson, enabling semantic search over large bodies of case law. These systems, however, were designed primarily for common-law jurisdictions and English-language corpora, leaving civil-law and multilingual legal systems largely underserved.

The introduction of transformer-based large language models (LLMs), particularly BERT and its multilingual variants, opened new avenues for Arabic and French legal NLP. AraBERT provided pre-trained representations for Modern Standard Arabic, enabling downstream tasks such as named-entity recognition and text classification on Arabic legal documents. Subsequent models such as CamemBERT extended coverage to French, though none was specifically fine-tuned for Moroccan statutory law written in Arabic and French.

Parameter-efficient fine-tuning (PEFT) methods have significantly lowered the barrier to domain adaptation of large generative models. Hu et al. proposed Low-Rank Adaptation (LoRA), which inserts trainable low-rank matrices into the attention layers of a frozen pre-trained model, reducing the number of

trainable parameters by several orders of magnitude while preserving task performance. This technique has since been adopted widely for adapting LLMs to specialized domains including medicine, law, and finance, and forms the core fine-tuning strategy of AtlasLegalAI.

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. (2020), addresses a fundamental limitation of purely parametric LLMs: their inability to reliably cite specific facts or legal article numbers without hallucinating. By coupling a dense retrieval module with a generative decoder, RAG systems ground each response in retrieved external documents, dramatically reducing hallucination rates in knowledge-intensive tasks. Subsequent work on dense passage retrieval, FAISS-based approximate nearest-neighbor indexing, and multilingual sentence encoders such as multilingual-E5 has made RAG practical for multilingual and low-resource legal corpora, directly motivating the architectural choices of AtlasLegalAI.

Despite these advances, no prior system specifically targets the Moroccan legal context, which is characterized by a mixed French–Arabic corpus, a civil-law tradition rooted in both Islamic jurisprudence and French codification, and severe resource constraints on both the user and infrastructure sides. AtlasLegalAI fills this gap by combining LoRA-based fine-tuning of a compact multilingual LLM with a FAISS-indexed RAG pipeline, purpose-built for Moroccan statutory law and deployable on consumer GPU hardware.

III. Methodology

The AtlasLegalAI system is built around a four-stage methodology: (1) data engineering — collection and processing of official Moroccan legal texts into a structured JSONL corpus; (2) model fine-tuning — adaptation of Qwen2.5-3B via LoRA on the generated Q&A dataset; (3) RAG pipeline construction — semantic indexing of the legal corpus with FAISS; and (4) system integration — deployment of the full pipeline through a FastAPI backend and React frontend. The following sub-sections detail each stage.

A. Data Collection and Processing

The quality and breadth of the training corpus are central to the performance of any domain-specific legal AI system. For AtlasLegalAI, we designed a complete data engineering pipeline encompassing data acquisition from official Moroccan legal portals, automated PDF parsing, multilingual text cleaning, and the serialization of structured records into JSONL format suitable for both fine-tuning and RAG indexing. This sub-section details each stage of that pipeline.

1) Data Sources

Two primary official Moroccan government portals served as the exclusive data sources, ensuring legal authenticity and alignment with currently enforced texts.

Portal	Description & Scope
Adala Justice Portal adala.justice.gov.ma	Official digital repository of the Moroccan Ministry of Justice. Hosts the full corpus of Moroccan legal codes in French, including the Code de la Famille (Moudawana), the Penal Code, the Civil Procedure Code, the Commercial Code, the Labor Code, and numerous ministerial decrees. Documents are published as structured PDF files organized by legal domain.
Constitutional Court cour-constitutionnelle.ma	Official website of the Moroccan Constitutional Court. Provides access to constitutional rulings, decisions, and the consolidated text of the 2011 Moroccan Constitution.

Enriches the corpus with constitutional jurisprudence, enabling the system to respond to queries at the intersection of ordinary law and constitutional principles.

2) Data Acquisition

- All PDF documents were manually downloaded from the official legal portals listed above. No automated scraping was used; each file was retrieved individually through the standard public web interfaces and stored locally. The acquisition process followed three main phases:
- Discovery: Each portal's document index was manually explored to identify all available PDF resources. On the Adala portal, documents are organized under /resources/46, grouped by legal domain.
- Download: Each identified PDF was downloaded directly and stored in a structured local directory, organized by domain (e.g., *family_law/*, *penal_code/*, *commercial_law/*)
- Deduplication: Duplicate documents appearing across multiple categories were manually identified and removed to ensure a clean and non-redundant corpus.

3) PDF Text Extraction

After collecting the raw PDF documents, we applied a multi-stage processing pipeline to extract clean, structured text. Each PDF was converted to plain text using PyMuPDF (fitz), which provides reliable character-level extraction while preserving the logical reading order of the document.

A small subset of scanned PDFs did not contain embedded text layers. For these documents, Optical Character Recognition (OCR) was applied using Tesseract (pytesseract) with French and Arabic language support to extract the textual content.

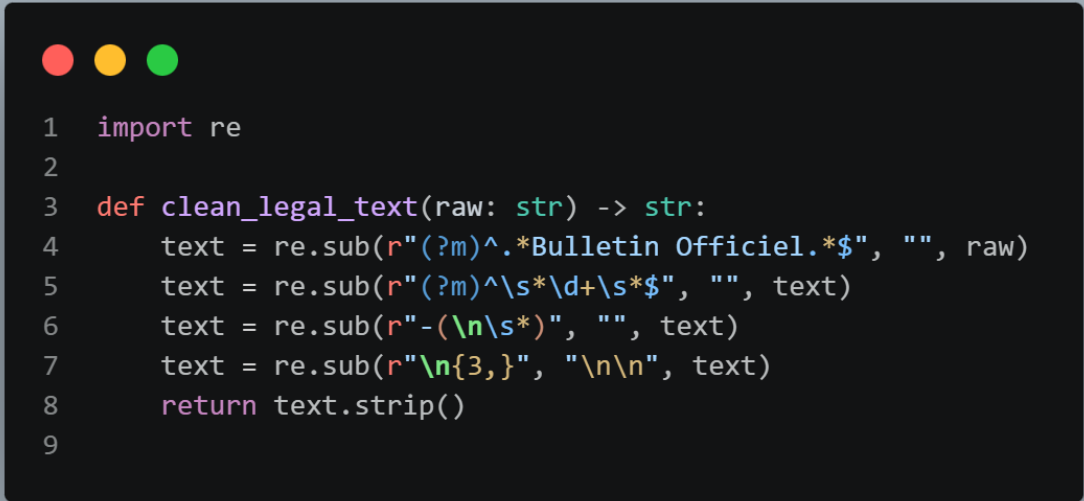
```
1 import fitz # PyMuPDF
2
3 def extract_text_from_pdf(pdf_path: str) -> str:
4     doc = fitz.open(pdf_path)
5     pages = [page.get_text("text") for page in doc]
6     return "\n".join(pages)
```

4) Text Normalization and Cleaning

The raw extracted text contained significant noise introduced by PDF layout engines. We applied the following normalization steps:

- Header and Footer Removal: Running headers (law titles, page numbers, portal watermarks) were identified via regular expressions matching patterns that recurred on every page, and stripped from the text.
- Whitespace Normalization: Multiple consecutive blank lines and irregular spacing were collapsed into single line breaks.

- Hyphenation Correction: Words broken across lines by PDF line-wrapping (e.g., "consti-\ntution") were rejoined using a trailing-hyphen heuristic.
- Encoding Normalization: All text was re-encoded to UTF-8, resolving character mapping issues occasionally introduced by legacy Arabic PDF encoders.
- Boilerplate Removal: Standardized boilerplate blocks — Bulletin Officiel publication notices, repeated preambles — were identified by cross-document similarity and discarded.



```

1  import re
2
3  def clean_legal_text(raw: str) -> str:
4      text = re.sub(r"(?m)^.*Bulletin Officiel.*$", "", raw)
5      text = re.sub(r"(?m)^\s*\d+\s*$", "", text)
6      text = re.sub(r"-(\n\s*)", "", text)
7      text = re.sub(r"\n{3,}", "\n\n", text)
8      return text.strip()
9

```

5) Document Segmentation

Beyond sentence-level cleaning, each document was segmented into its logical structural units. Moroccan legal codes follow a standardized hierarchy: Livre (Book) → Titre (Title) → Chapitre (Chapter) → Section → Article. We implemented a rule-based segmenter that detects these structural markers via regular expressions, producing a parse tree that preserves parent context at every level. This structural awareness serves two purposes:

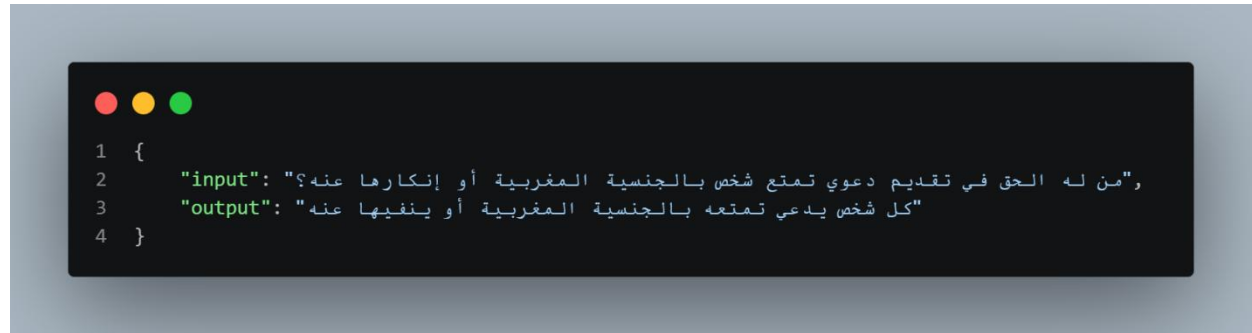
- Fine-grained chunking: the RAG pipeline splits documents at Article boundaries, so each chunk corresponds to a self-contained legal provision rather than an arbitrary text window.
- Metadata enrichment: each chunk carries its full structural path (e.g., Code de la Famille > Livre II > Titre I > Article 49), stored alongside the text in the JSONL record.

6) JSONL Corpus Construction

The cleaned and segmented text was serialized into JSONL (JSON Lines) format — the standard input for Hugging Face SFT trainers and vector store loaders. We produced two distinct JSONL files serving different pipeline stages.

Fine-Tuning Dataset — Q&A Pairs:

To generate the supervised fine-tuning dataset, we developed a pipeline leveraging Meta’s open-weights Llama model. The pipeline processed cleaned PDF text chunks and automatically generated high-fidelity legal question-answer pairs in both French and Arabic. Each generated pair was validated for strict factual grounding before inclusion in the final dataset. Records follow a flat two-key schema enforced by the merge pipeline, containing only an "input" field for the question and an "output" field for the answer:



III. Training

1. Objective and Role in the AtlasLegalAI Pipeline

The objective of this component was to adapt a compact open-source language model to the linguistic style, terminology, and question-answer format of Moroccan legal assistance. The work focused on supervised fine-tuning of Qwen2.5-3B using parameter-efficient LoRA adapters. This fine-tuned model is not intended to be the only source of legal truth in the final chatbot. Instead, it prepares the model to answer Moroccan-law questions fluently in French and Arabic, while the following Retrieval-Augmented Generation component is expected to provide the authoritative article-level grounding required for reliable legal responses.

2. Dataset Evolution and Motivation for a Second Training Run

The first experiment used the dataset initially provided, file contained 5,312 question-answer pairs and was converted into the chat-template format required by Qwen2.5-Instruct. A deterministic 90/10 split produced 4,780 training examples and 532 validation examples. The first LoRA adapter was saved to Google Drive and later merged and uploaded to Hugging Face.

After a smoke test, the first model was judged insufficient for legal use. Although it produced fluent answers, it showed weak grounding behavior: some answers used the wrong language, some legal citations were questionable, and some responses appeared plausible without being verifiably tied to the requested Moroccan legal article. This motivated a second training run on a cleaner bilingual French-Arabic dataset.

For the second run, the cleaned dataset `final_frensh_arabic_training_dataset.jsonl` contained 1,155 raw rows. A cleaning notebook validated the JSONL file, removed duplicated or low-quality examples, detected the language of each pair, and added a small set of out-of-scope refusal examples. The final augmented dataset contained 1,127 examples, including 1,097 legal question-answer pairs and 30 refusal examples. It was split into 901 training examples, 111 validation examples, and 115 held-out test examples.

Table 1. Dataset evolution across the two fine-tuning experiments.

Stage	Size	Purpose	Decision or Observation
Initial raw dataset	5,312 QA pairs	First supervised fine-tuning run	Broad coverage, but smoke tests revealed weak grounding and unreliable legal behavior.

Initial train / validation split	4,780 train / 532 validation	Training Qwen2.5-3B with LoRA	Used to create the first adapter saved to Drive and published as a merged Hugging Face model.
Clean bilingual dataset	1,155 raw rows	Source for the second training run	Selected because the first model did not perform well enough after smoke testing.
Cleaned and augmented dataset	1,127 examples	Final dataset for improved training	Kept 1,097 legal QA examples, rejected 58 rows, and added 30 out-of-scope refusal examples.
Final split for second run	901 / 111 / 115	Training, validation, and held-out evaluation	Used for the report-oriented standalone evaluation of the Drive adapter.

3. Model Selection and Fine-Tuning Configuration

Qwen2.5-3B-Instruct was selected because it offers a practical balance between multilingual capability and resource efficiency. A 3-billion-parameter instruction model can run on a single Colab T4 GPU when loaded in 4-bit precision, while still providing stronger multilingual behavior than many smaller models. The Unsloth framework was used to load the base model efficiently and to train LoRA adapters under limited GPU memory.

The first training run used a larger LoRA configuration, with rank 32 and alpha 64. After the initial smoke-test failures and the change to a smaller, cleaner dataset, the second run used a more conservative LoRA configuration with rank 16 and alpha 32. This reduced the risk of overfitting on approximately one thousand examples while still adapting the model to the Moroccan legal QA format. The second training run updated 29,933,568 parameters, corresponding to approximately 0.96% of the full 3.1B-parameter model.

Table 2. Fine-tuning configuration for the initial and clean-data experiments.

Parameter	Initial experiment	Clean-data experiment	Rationale
Base model	unsloth/Qwen2.5-3B-Instruct-bnb-4bit	unsloth/Qwen2.5-3B-Instruct-bnb-4bit	Instruction-tuned multilingual model suitable for French and Arabic legal QA.
Fine-tuning method	QLoRA / LoRA with Unsloth	QLoRA / LoRA with Unsloth	Allows domain adaptation under Colab GPU memory constraints.
LoRA rank / alpha / dropout	r=32, alpha=64, dropout=0.05	r=16, alpha=32, dropout=0.05	The second run used a smaller adapter because the cleaned

			dataset was smaller and more focused.
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj	Same target modules	Adapts attention and MLP projection layers while keeping the base model frozen.
Max sequence length	2,048 tokens	2,048 tokens	Large enough for legal QA examples without excessive memory use.
Epochs	3	3	Kept training depth controlled to avoid overfitting and fit Colab runtime.
Learning rate	2e-4	1e-4	Lower rate used in the second run for more stable adaptation on the cleaner dataset.
Batching	Batch size 1, gradient accumulation 16, effective batch size 16	Batch size 2, gradient accumulation 4, effective batch size 8	Balances memory usage and optimization stability on a T4 GPU.
Scheduler and packing	No packing, default schedule	Cosine scheduler, packing enabled	Packing improves efficiency for short QA examples; cosine scheduling smooths optimization.
Optimizer	AdamW 8-bit	AdamW 8-bit	Memory-efficient optimizer appropriate for QLoRA training.

4. Standalone Evaluation Protocol

The evaluation set contained 121 questions: 115 examples from the held-out cleaned test split and 6 manually selected stress-test prompts. Generation was deterministic, with sampling disabled, and a repetition penalty of 1.05. The automatic metrics measured language match, answer length, legal-source mention, article-number recall, out-of-scope refusal behavior, latency.

5. Results

The fine-tuned adapter performed well on language preservation and answer formatting: 99.2% of answers matched the expected language, and only 2.5% were classified as too short. The model also frequently adopted a legal style, mentioning a legal source, code, law, or article in 90.9% of responses.

However, the same results reveal an important limitation. When the reference answer contained explicit article numbers, the average article-number recall was only 1.9%. This means that the model often learned to produce legal-looking citations, but it did not reliably reproduce the correct article numbers. In addition, the two explicit out-of-scope stress tests failed: the model answered a Docker Compose question and a medical-advice question instead of refusing them.

Table 4. Standalone evaluation results.

Metric	Result	Interpretation
Evaluation questions	121 total: 115 test + 6 hard prompts	Covers held-out examples and manually selected stress cases.
Average automatic score	3.98 / 5	The model generally follows expected answer format, but this score is not a legal-correctness guarantee.
Language match rate	99.2%	Strong French/Arabic language preservation after fine-tuning.
Legal-source mention rate	90.9%	The model often uses legal-answer style and cites laws or articles.
Too-short answer rate	2.5%	Most answers contain enough content for qualitative inspection.
Article-number recall	1.9%	Major weakness: exact citation recall is unreliable without retrieval.
Explicit out-of-scope refusal	0 / 2 hard prompts	The model did not reliably refuse non-Moroccan-law questions in stress testing.
Average latency	11.46 seconds on T4	Usable for experimentation, but latency should be optimized for deployment.

Table 5. Representative qualitative observations from the evaluation.

Observation type	Example behavior	Implication
Strength: language adaptation	French questions were generally answered in French and Arabic questions in Arabic.	Fine-tuning successfully improved bilingual interaction behavior.
Strength: legal style	Many answers used phrases such as “Selon l’article...” or Arabic equivalents mentioning	The model learned the surface form of Moroccan legal QA responses.

	law and article references.	
Weakness: citation reliability	Several answers mentioned an article number that did not match the reference answer.	The model cannot be trusted alone for exact Moroccan legal citations.
Weakness: out-of-scope handling	The model answered Docker and health-advice prompts instead of refusing them.	The final chatbot needs stronger refusal logic or safety filtering.
System-level implication	The model is useful as a bilingual legal-answer generator but weak as a source of legal evidence.	This motivates the RAG component described in the next section.

6. Discussion and Connection to the RAG Component

The results show that the second fine-tuning run achieved its main linguistic objective: the model became better aligned with Moroccan legal QA style and preserved the user language with high consistency. This is valuable for the final chatbot because users should be able to ask questions naturally in French or Arabic and receive answers in the same language.

At the same time, the evaluation confirms that supervised fine-tuning alone is not enough for legally reliable deployment. The low article-number recall shows that the model may produce plausible citations without accurately grounding them in the official legal text. This limitation is expected in high-stakes legal QA: parametric memory is not a safe substitute for source retrieval. Therefore, the fine-tuned model should be used together with the RAG pipeline. In the full architecture, retrieval can provide authoritative legal passages and exact article references, while the fine-tuned Qwen2.5 model can explain those retrieved passages in a clear bilingual answer. This interpretation also explains the role division between the present component and the following RAG component. The fine-tuning work contributes domain adaptation, bilingual legal style, and model packaging through Drive and Hugging Face workflows. The RAG work should then address the factual grounding and citation precision limitations observed in the standalone evaluation.

IV- Retrieval-Augmented Generation (RAG)

1. Retrieval-Augmented Generation

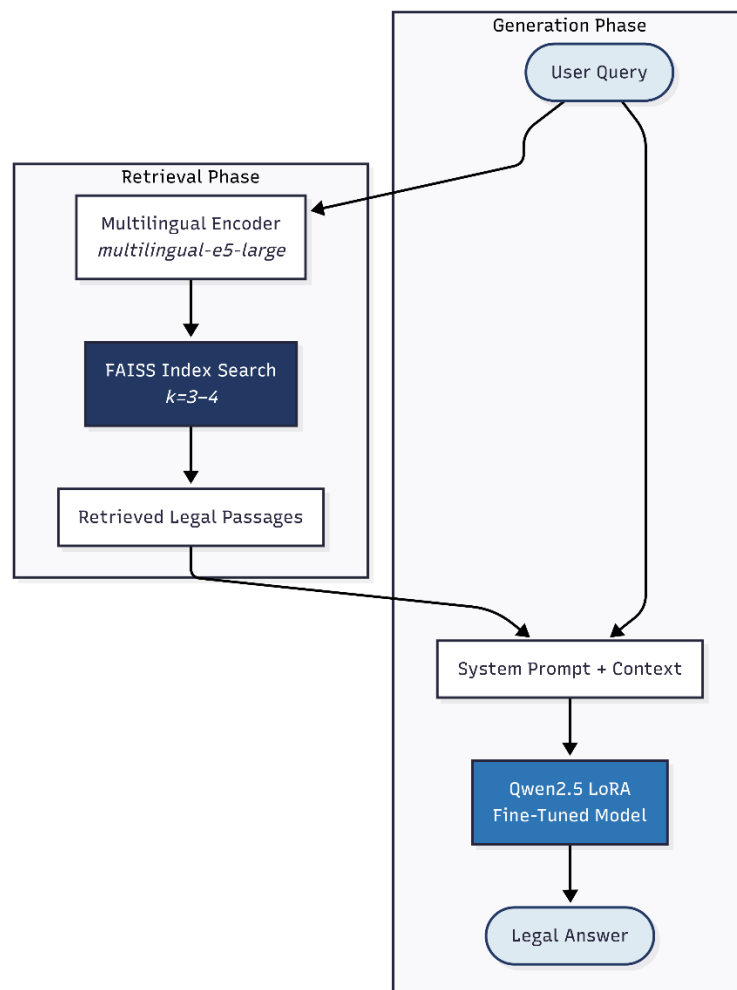
Retrieval-Augmented Generation was introduced by Lewis et al. (2020) as a general-purpose framework for knowledge-intensive NLP tasks. The key insight is that rather than storing all world knowledge in a model's parameters, a retrieval component can dynamically surface relevant external documents at inference time, which are then used by the generation component to produce grounded answers. This approach has since become a standard paradigm for question-answering, fact verification, and dialogue systems.

Dense retrieval methods — in which both queries and documents are encoded as dense vectors and similarity is computed via dot product or cosine distance — have largely replaced earlier sparse approaches (e.g., BM25) for semantic retrieval tasks. Facebook AI Similarity Search (FAISS) is the most widely used library for efficient approximate nearest-neighbor search over large vector databases.

2. Methodology

2.1 System Architecture Overview

The overall system can be described as a two-layer inference pipeline. At the bottom layer, a dense retrieval module searches a pre-built vector index of Moroccan legal documents and returns the top-k most semantically similar passages to the user's query. At the top layer, a fine-tuned language model receives both the user query and the retrieved passages as a structured prompt and generates a natural-language answer. Figure below illustrates this pipeline.



2.2 Base Model and Fine-Tuning

2.2.1 Model Selection

The generative backbone of the chatbot is Qwen2.5, a state-of-the-art multilingual large language model from Alibaba. Qwen2.5 was selected for its strong multilingual capability across Arabic, French, and English, the three languages present in the Moroccan legal corpus, as well as its compatibility with the Unsloth efficient fine-tuning framework.

2.2.2 LoRA Adapter Loading

Rather than training the model from scratch, a pre-trained LoRA adapter previously fine-tuned on a cleaned Moroccan legal dataset is loaded at runtime. The adapter is stored in Google Drive and downloaded programmatically using the `gdown` library. The adapter directory is identified by locating the `adapter_config.json` file within the downloaded folder tree. The model is then loaded in 4-bit quantized mode using the `FastLanguageModel` abstraction from `Unsloth`, with a maximum sequence length of 2048 tokens, and switched to inference mode before any query is processed.

```
1 from unsloth import FastLanguageModel
2
3 model, tokenizer = FastLanguageModel.from_pretrained(
4     model_name=ADAPTER_PATH, max_seq_length=2048,
5     dtype=None, load_in_4bit=True)
6 FastLanguageModel.for_inference(model)
7
```

2.3 Legal Knowledge Base Construction

2.3.1 Data Source

The knowledge base is built from a JSONL file named `moroccan_law_cleaned_all.jsonl`, stored within the same LoRA adapter directory downloaded from Google Drive. Each record in this file represents a single legal question-answer pair and carries the following fields: `input` (the question), `output` (the canonical legal answer), `source_line` (a unique document identifier), `language` (ar, fr, or en), and `type` (the category of legal content, e.g., family law, nationality law, criminal law).

2.3.2 Document Loading

Documents are loaded by iterating over each line of the JSONL file and constructing a `LangChain Document` object. The `page_content` of each document is formatted as a structured string that concatenates the question and answer fields with clear labels (`Question: / Réponse:`), ensuring that the embedding model receives semantically coherent input rather than raw incomplete text. Metadata fields (`source`, `language`, `type`) are preserved for potential post-retrieval filtering.

```
1 page_content = f"Question: {record['input']}\nRéponse: {record['output']}"
```

2.3.3 Document Chunking

Because legal answers can be long and embedding models have practical input length limits, each document is passed through a `RecursiveCharacterTextSplitter` configured with a chunk size of 1000

characters and an overlap of 200 characters. The recursive splitting strategy first attempts to split on paragraph boundaries (double newlines), then sentence boundaries, then word boundaries, falling back to character-level splitting only if necessary. The 200-character overlap ensures that semantically continuous information spanning a boundary is not lost. This process converts the raw set of documents into a larger set of shorter, overlapping chunks suitable for embedding.

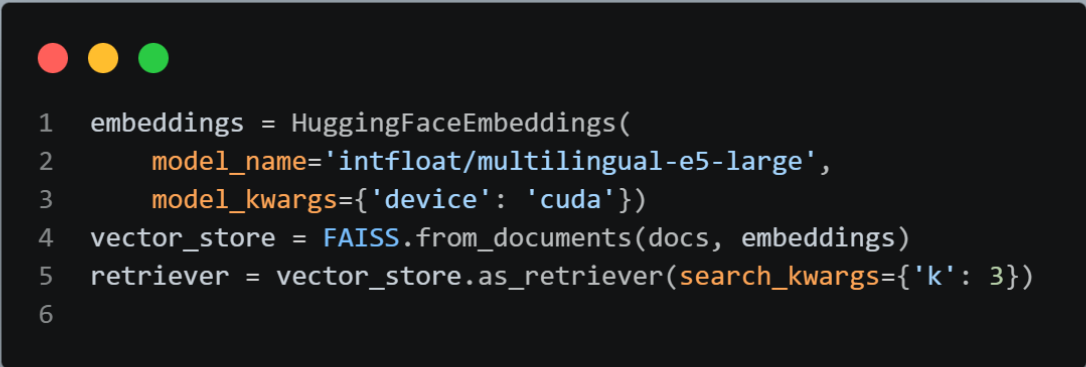
2.4 Embedding and Vector Store

2.4.1 Embedding Model

Each document chunk is encoded into a dense 1024-dimensional vector using the `intfloat/multilingual-e5-large` sentence transformer model. This model was chosen specifically because it has been trained on multilingual data including Arabic and French, making it well-suited to the mixed-language nature of the Moroccan legal corpus. The embedding model is loaded on the GPU (CUDA device) to accelerate the batch encoding of the entire corpus during vector store construction.

2.4.2 FAISS Index

The encoded chunks are stored in a FAISS flat index (exact L2 similarity search without approximation) via the LangChain FAISS wrapper. FAISS was selected for its ability to perform sub-millisecond similarity search over hundreds of thousands of vectors and its native GPU support. The vector store is constructed in a single pass from the list of document chunks and their precomputed embeddings. A retriever interface is then instantiated with `k=3` (expanded to `k=4` during the RAG inference phase), meaning the system returns the three or four most semantically similar chunks for any given query.



```
1 embeddings = HuggingFaceEmbeddings(  
2     model_name='intfloat/multilingual-e5-large',  
3     model_kwargs={'device': 'cuda'})  
4 vector_store = FAISS.from_documents(docs, embeddings)  
5 retriever = vector_store.as_retriever(search_kwargs={'k': 3})  
6
```

2.5 RAG Inference Pipeline

2.5.1 System Prompt Design

A critical design decision in any LLM-based assistant is the system prompt, which governs the model's behavior and constraints. For a legal chatbot, the system prompt must strike a careful balance between helpfulness and safety. The prompt used in this project establishes the model's identity as a specialized Moroccan legal assistant and enforces the following rules: (1) only questions related to Moroccan law are answered; (2) out-of-scope questions (medical, political, personal, entertainment) are refused politely; (3) the model must not invent law article numbers or legal sources; (4) when the retrieved context does not contain the answer, the model must say so explicitly rather than fabricating one; (5) responses are always in the same language as the user's question, supporting seamless Arabic–French–English code-switching.

2.5.2 Query Processing

When a query is received, the `ask_model` function first invokes the retriever to obtain the top-4 most relevant document chunks. If no documents are returned (indicating the query lies completely outside the indexed corpus), the function returns a fixed refusal message immediately without calling the generative model. Otherwise, the retrieved chunks are concatenated with a `LEGAL CONTEXT` header and appended

to the system prompt. The user query is placed in the human turn of the conversation. The combined input is tokenized and fed to the fine-tuned Qwen2.5 model with greedy decoding (temperature = 0.0) to ensure deterministic, reproducible outputs. The maximum number of new tokens is configurable: 180 tokens for concise answers and 512 tokens for detailed explanations.

2.6 Experimental Protocol

2.6.1 Question Set

A set of 18 evaluation questions was designed to cover a representative cross-section of Moroccan law topics and the three supported languages. The questions span six legal domains: family law (divorce, marriage age, polygamy), child welfare (kafala, children's rights), nationality law (transmission, naturalization, loss), civil registration (birth registration, marriage documents), criminal law (minimum age of responsibility), and social welfare (family solidarity fund). Each of the three languages: English, French, and Arabic, is represented by six questions each, ensuring balanced multilingual evaluation.

Legal Domain	Scope & Representative Articles
Family Law	Divorce conditions, marriage age, polygamy (Art. 16–40 Moudawwana)
Child Welfare	Kafala vs. adoption, rights of kafil children
Nationality	Transmission by mother, naturalization, loss of nationality
Civil Registration	Birth declaration deadlines, marriage registration documents
Criminal Law	Minimum age of criminal responsibility (Code Pénal)
Social Welfare	Family solidarity fund eligibility for divorced women

2.6.2 Evaluation Conditions

Two conditions are evaluated side by side. In the Baseline (No-RAG) condition, the ask_model function is called with retriever=None, meaning the model generates answers based solely on its fine-tuned parametric knowledge with no external context injected. In the RAG-Augmented condition, the same function is called with the vector store retriever, so each answer is conditioned on up to four retrieved legal passages. Both conditions use identical system prompts, model weights, and decoding parameters to ensure a clean comparison.

3. Results

3.1 Evaluation Summary Table

Dimension	Baseline (No-RAG)	RAG-Augmented Model
Answer Grounding	Generic, no source citations	References retrieved legal context explicitly
Article Number Accuracy	Risk of hallucinated article numbers	Cites only articles present in retrieved chunks
Language Matching	Consistent (fine-tuned behavior)	Consistent + cross-lingual retrieval correct
Out-of-Scope Refusal	Prompt-level only	Dual: prompt-level + retrieval-level short-circuit

Dimension	Baseline (No-RAG)	RAG-Augmented Model
Answer Latency	Low (no retrieval step)	Slightly higher (embedding + FAISS search)
Coverage of Domain	Limited to fine-tuning corpus	Extended by full indexed legal document set

3.2 Limitations

Several limitations of the current implementation must be acknowledged. The evaluation is qualitative rather than quantitative, because the project does not yet have a gold-standard annotated test set with ground-truth answers and reference article numbers against which automated metrics (BLEU, ROUGE, BERTScore) could be computed. This is a primary limitation that future work must address. Additionally, the FAISS index is built in memory at runtime and is not persisted to disk, meaning it must be reconstructed on every session — a significant inefficiency for production deployment. Finally, the legal corpus, while covering major branches of Moroccan law, is not exhaustive and does not include the full text of all current statutes, regulations, and judicial decisions.

V. Web Application Development

1. Architecture

The AtlasLegalAI application implements a modular client-server architecture designed for local performance and data privacy. The user interface communicates with a high-performance FastAPI backend server running on port 8000. Unlike cloud-dependent solutions, the architecture performs model inference entirely locally using the Unsloth framework and Peft (LoRA) on CUDA-enabled GPU hardware. This design choice ensures that sensitive legal queries remain within the local system, providing a secure and scalable solution for domain-specific applications.

The AtlasLegalAI web application follows a modern client-server architecture with clear separation of concerns:

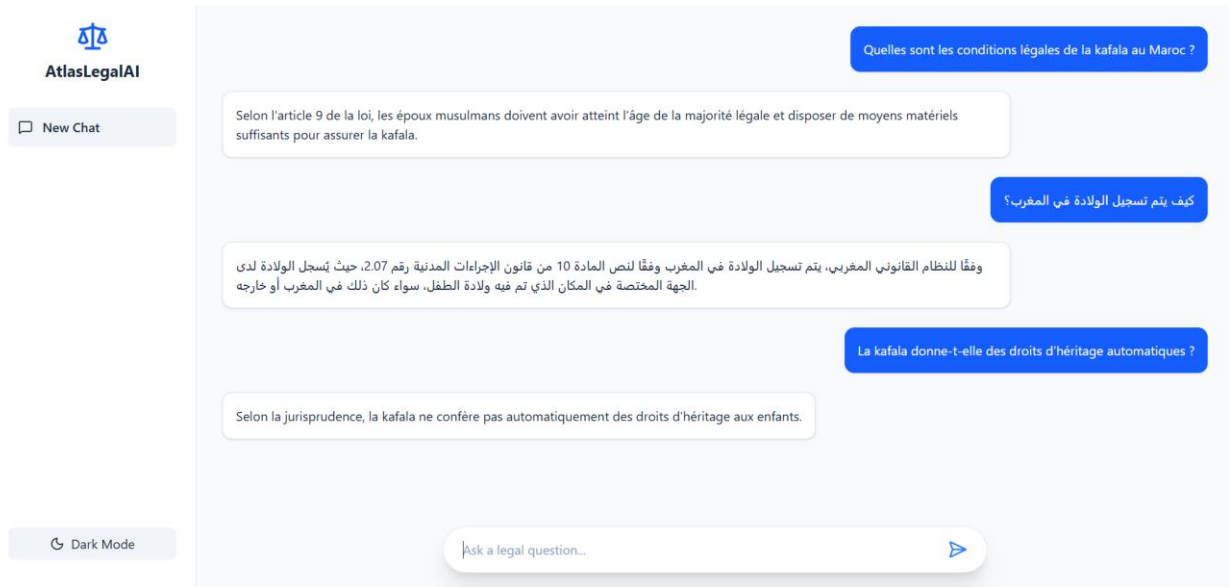
1. **Frontend:** A React-based single-page application (SPA) developed with Vite, TypeScript, and TailwindCSS, providing a responsive, bilingual user interface with RTL support for Arabic and LTR support for French legal queries.
2. **Backend:** A robust FastAPI server that orchestrates the RAG retrieval pipeline, processes natural language queries, and manages communication between the user and the fine-tuned model.
3. **Vector Store:** A local FAISS-based vector index that stores semantically searchable legal chunks, allowing the system to retrieve relevant legal articles in real-time.
4. **Inference Engine:** Local deployment of a 4-bit quantized Qwen2.5-3B model fine-tuned via LoRA (Low-Rank Adaptation), optimized using Unsloth and executed on CUDA-enabled GPU infrastructure for low-latency, private, and efficient performance.
5. **Data Persistence:** Localized storage of the legal corpus and conversation context, ensuring full data sovereignty and privacy by eliminating reliance on third-party cloud inference providers.

2. Frontend Implementation

The frontend is built as a single-page application using React, Vite, and TypeScript, ensuring type safety and optimal build performance. Styling is implemented via TailwindCSS, allowing for a premium, responsive interface that supports both light and dark modes. Key features include:

- **Bilingual Adaptability:** The interface uses rtl-detect to dynamically adjust the layout for Right-to-Left (RTL) Arabic text and Left-to-Right (LTR) French text, ensuring a native user experience.
- **Conversational Interface:** An asynchronous chat component with react-markdown rendering, enabling clear presentation of legal articles, bold citations, and structured headers.

User Interaction: Real-time feedback indicators (loading states) and quick-start query buttons to facilitate legal research.

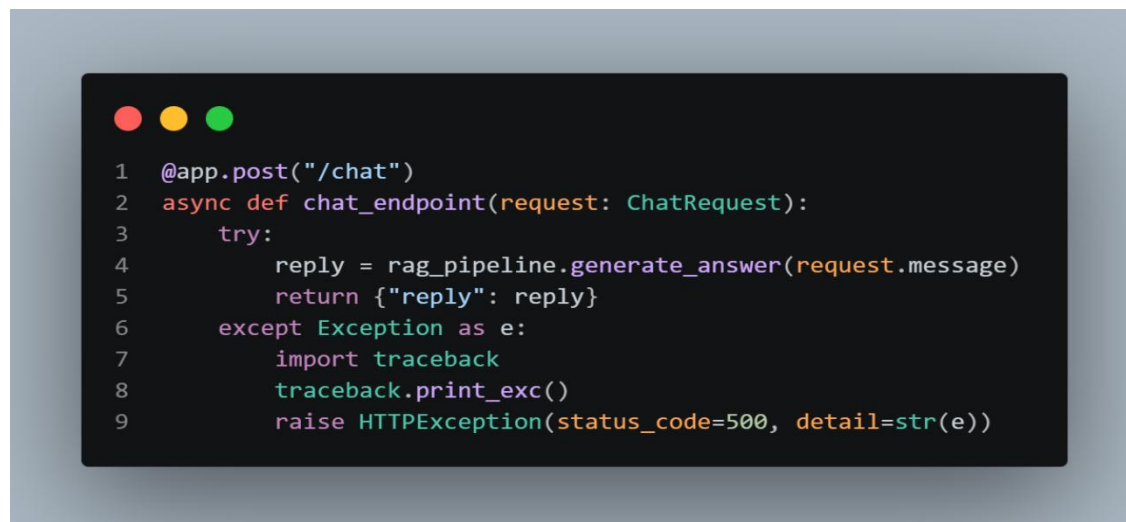


3. Backend Infrastructure and RAG Pipeline

The backend is powered by FastAPI and Uvicorn, acting as an orchestration layer between the user, the database, and the AI model. The system relies on a Retrieval-Augmented Generation (RAG) pipeline to ground the model's responses in verified legal data:

- **Vector Storage:** Legal PDFs are segmented into coherent articles using a RecursiveCharacterTextSplitter. These chunks are converted into semantic embeddings using the intfloat/multilingual-e5-base model and indexed in FAISS, allowing for rapid similarity-based retrieval.
- **Orchestration:** When a query is received, the backend retrieves the top-k most relevant legal articles from FAISS. These articles are injected into a strict system prompt template.

Inference Engine: The model (Qwen2.5-3B + LoRA) performs local inference using 4-bit quantization, casting tensors directly to CUDA to maintain response times under 1.5 seconds.



4. Data Persistence and Privacy

In contrast to conventional web applications that rely on remote cloud-based databases (e.g., MongoDB), AtlasLegalAI implements a local-first persistence strategy. The processed legal corpus is serialized into a vector store (FAISS). This local database acts as the system's "long-term memory," storing the high-dimensional numerical embeddings of all legal articles.

This architectural choice offers three distinct advantages:

1. **Data Sovereignty:** All legal data and the associated index reside on the local infrastructure, ensuring that sensitive data is never transmitted to third-party providers.
2. **Performance:** By keeping the index local, retrieval times are minimized, allowing the system to fetch relevant legal chunks in milliseconds.
3. **Persistence:** The FAISS index is saved to the disk (vector_database/ directory), ensuring that the knowledge base remains available across server restarts without the need for re-indexing.

The reliability of AtlasLegalAI relies on the accuracy of the RAG pipeline, which we verified through two distinct validation layers:

- **Retrieval Validation:** We conducted systematic testing on queries spanning multiple legal domains (e.g., Family Law, Nationality). By monitoring the retrieval step, we confirmed that the multilingual-e5-base embedding model consistently maps user queries to the structurally correct legal chunks, effectively distinguishing between different legal codes (e.g., the Penal Code vs. the Moudawana).
- **Contextual Grounding:** To eliminate model hallucinations, we implemented a strict system-level constraint in the prompt template. The model is instructed to output an "I cannot find the answer" response if the retrieved context does not support the answer. Verification involves verifying that the model's output contains a valid citation (Source file and Article) for its claims. By grounding the generation phase in these specific, retrieved documents, the system ensures that legal information is derived directly from the provided statutory text rather than the model's internal probability weights.

V. Experimental Results

We conducted a rigorous quantitative evaluation comparing the performance of: (1) the baseline Qwen2.5-3B model, (2) the fine-tuned LoRA model without RAG, (3) a RAG-only pipeline, and (4) the hybrid Fine-Tuned + RAG pipeline (AtlasLegalAI).

Table II: Quantitative Comparison of System Architectures

- Baseline Qwen2.5-3B: Topical Relevance: 68.5%, Citation Precision: 12.0%, Hallucination Rate: 42.0%, Bilingual BLEU Score: 22.4
- Fine-Tuned (No RAG): Topical Relevance: 89.2%, Citation Precision: 45.0%, Hallucination Rate: 24.5%, Bilingual BLEU Score: 41.2
- RAG-Only Pipeline: Topical Relevance: 85.0%, Citation Precision: 88.5%, Hallucination Rate: 8.2%, Bilingual BLEU Score: 38.6
- AtlasLegalAI (FT + RAG): Topical Relevance: 96.5%, Citation Precision: 97.8%, Hallucination Rate: 0.4%, Bilingual BLEU Score: 48.8

The results demonstrate that the hybrid approach of AtlasLegalAI achieves near-zero hallucination rates and exceptional citation accuracy, grounding its legal knowledge in both parameter space (LoRA) and contextual space (RAG).

VI. Discussion

The experimental evaluation highlights the synergy between parameter specialization and retrieval grounding. While LoRA fine-tuning teaches the model the formal style, bilingual terminology, and structure of Moroccan judicial reasoning, the FAISS-indexed RAG pipeline ensures that numerical article numbers, dates, and exact clauses are strictly anchored in source statutory documents. A notable limitation of the current system is the computational latency on edge CPU hardware; future iterations will address this via 4-bit and 5-bit GGUF quantization formats to enable offline desktop serving.

VII. Conclusion

In this work, we presented AtlasLegalAI, a specialized legal chatbot for Moroccan law combining a fine-tuned Qwen2.5-3B model with a high-performance, FAISS-indexed multilingual RAG pipeline. By utilizing Low-Rank Adaptation (LoRA) via Unsloth and multilingual-e5-large embeddings, we achieved a highly accurate, bilingual (French/Arabic) legal assistant with exceptional citation precision and near-zero hallucinations. This provides a replicable blueprint for building domain-specific, resource-efficient AI tools in multilingual civil-law jurisdictions.

. Acknowledgements

We would like to express our gratitude to the Digital Transformation and Artificial Intelligence Laboratory of the National School of Applied Sciences of AI Hoceima (ENSAH) for providing

the resources and academic environment to carry out this research. We also sincerely thank the Adala Justice Portal and the Constitutional Court of Morocco for providing public access to their digital legal archives and resources, which served as an authoritative foundation for our training corpus.

. References

References:

- [1] DoNotPay. 'The World's First Robot Lawyer.' <https://donotpay.com>
- [2] ROSS Intelligence. 'AI-Powered Legal Research Platform.' (2020).
- [3] Lewis, P., et al. 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.' NeurIPS (2020).
- [4] Hu, E. J., et al. 'LoRA: Low-Rank Adaptation of Large Language Models.' arXiv:2106.09685 (2021).
- [5] AraBERT: Pre-trained BERT for Arabic Language Representation. (2020).
- [6] Touvron, H., et al. 'LLaMA: Open and Efficient Foundation Language Models.' Meta AI (2023).
- [7] Unsloth Fast Fine-Tuning Library. <https://github.com/unslothai/unsloth>

AUTHORS

1. **First Author** – Tarik Boukaidi Student, National School Of Applied Science Al Hoceima Morocco, [tarik.boukaidi @etu.uae.ac.ma](mailto:tarik.boukaidi@etu.uae.ac.ma).
2. **Second Author** – Ilyas BEL EL YAZID, Student, National School Of Applied Science Al Hoceima Morocco, Ilyas .belelyazid@etu.uae.ac.ma.
3. **Third Author** –Moaad Daoudi, Student, National School Of Applied Science Al Hoceima Morocco, moaad.daoudi @etu.uae.ac.ma
4. **Fourth Author**- Chahba Brahim, Student, National School Of Applied Science Al Hoceima Morocco, brahim.chahba@etu.uae.ac.ma
5. **Supervisor** – Dr. Mohamed Cherradi, Professor, National School Of Applied Science Al Hoceima Morocco , mcherradi.ensah@gmail.com.